

Tutorial 3 Week 5

Tutorial Team 26

A0327301U Chao Chien Hung
A0274850Y Yao Tianhao

September 8, 2026

Problem 1: Nova the Mars Rover

We denote c = charged and l = low for brevity, and list policies in the order $[\pi(c), \pi(l)]$. The model is defined as follows:

State	Action	$P(c \mid s, a)$	$P(l \mid s, a)$	$R(s, a)$
Charged	Eco	0.95	0.05	7
Charged	Boost	0.70	0.30	10
Low	Eco	0.50	0.50	0
Low	Boost	0.10	0.90	2

Although the course generally assumes rewards are given upon entering a state, this problem explicitly specifies rewards based only on the current state and action. We therefore use $R(s, a, s') = R(s, a)$ throughout Problem 1. For $0 \leq \gamma < 1$, the value function is given by:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \mid S_0 = s \right].$$

Part 1: Discounting

(a) Role of Discounting

Discounting controls the trade-off between immediate rewards and long-term battery conservation. A reward k steps ahead is scaled by γ^k . For bounded rewards, setting $\gamma < 1$ also ensures the infinite sum remains finite:

$$|V^\pi(s)| \leq \frac{R_{\max}}{1 - \gamma}.$$

For instance, with $\gamma = 0.6$, a reward five steps ahead retains only $0.6^5 \approx 0.078$ of its original weight. However, with $\gamma = 0.9$, its weight is $0.9^5 \approx 0.59$. Consequently, an agent with $\gamma = 0.6$ is relatively short-sighted, whereas one with $\gamma = 0.9$ places a greater emphasis on preserving or restoring its battery. The corresponding effective horizons $1/(1 - \gamma)$ are approximately 2.5 and 10 steps.

Nevertheless, a higher discount factor does not necessarily alter the optimal action at every state. As shown in part (b), **both $\gamma = 0.6$ and $\gamma = 0.9$ yield the same optimal policy: [boost, eco]**.

(b) Optimal Policy as γ Varies

Defining the value difference between the two states as $\Delta = V_c - V_l$, the advantages of choosing eco over boost can be expressed as:

$$\begin{aligned} Q(c, \text{eco}) - Q(c, \text{boost}) &= -3 + 0.25\gamma\Delta, \\ Q(l, \text{eco}) - Q(l, \text{boost}) &= -2 + 0.40\gamma\Delta. \end{aligned}$$

Thus, eco is the greedy choice in the charged state exactly when $\gamma\Delta \geq 12$, and in the low state when $\gamma\Delta \geq 5$.

For a fixed policy with charged-state transition probabilities p_c, p_l and rewards r_c, r_l , subtracting the Bellman equations for the two states yields:

$$\Delta^\pi = (r_c - r_l) + \gamma(p_c - p_l)\Delta^\pi, \quad \implies \quad \Delta^\pi = \frac{r_c - r_l}{1 - \gamma(p_c - p_l)}.$$

We can now determine when each of the three possible deterministic policies is optimal:

- [boost, boost]: $\Delta = 8/(1 - 0.6\gamma)$. Both boost actions are preferred when $\gamma\Delta \leq 5$, yielding $\gamma \leq 5/11$.
- [boost, eco]: $\Delta = 10/(1 - 0.2\gamma)$. The policy is optimal when $5 \leq \gamma\Delta \leq 12$, yielding $5/11 \leq \gamma \leq 30/31$.
- [eco, eco]: $\Delta = 7/(1 - 0.45\gamma)$. Both eco actions are preferred when $\gamma\Delta \geq 12$, yielding $\gamma \geq 30/31$.

Summarizing these results (excluding the exact boundary values):

Discount factor	Unique optimal policy	Example
$0 \leq \gamma < 5/11$	[boost, boost]	$\gamma = 0.2$
$5/11 < \gamma < 30/31$	[boost, eco]	$\gamma = 0.8$
$30/31 < \gamma < 1$	[eco, eco]	$\gamma = 0.99$

The policy [eco, boost] is absent from this list because it requires both $\gamma\Delta \geq 12$ and $\gamma\Delta \leq 5$ simultaneously, which is impossible. Additionally, at $\gamma = 1$, the total-reward objective diverges for this continuing MDP, so this table strictly describes the finite-valued solutions for $\gamma < 1$.

(c) A Minimal-Entry Modification

Since adjusting γ alone cannot make [eco, boost] optimal due to the incompatible inequalities, we must modify the reward structure. Fixing $\gamma = 0.8$, we can achieve this by changing **only one reward entry**:

$$\boxed{R(l, \text{eco}) = -3.1 \quad (\text{instead of } 0).}$$

This introduces a penalty for operating the rover in eco mode when the battery is low, while leaving all transition probabilities and other rewards unchanged.

Under the target policy $\pi = [\text{eco}, \text{boost}]$, the evaluation equations are unaffected by this change because the policy does not select eco when low:

$$\begin{aligned} V_c &= 7 + 0.8(0.95V_c + 0.05V_l), \\ V_l &= 2 + 0.8(0.10V_c + 0.90V_l). \end{aligned}$$

Solving this system gives $V^\pi = [31.875, 16.25]$. The one-step action values using this continuation V^π are:

$$\begin{aligned} Q(c, \text{eco}) &= 31.875, & Q(c, \text{boost}) &= 31.75, \\ Q(l, \text{eco}) &= -3.1 + 0.8(0.5 \cdot 31.875 + 0.5 \cdot 16.25) = 16.15, & Q(l, \text{boost}) &= 16.25. \end{aligned}$$

Since $31.875 > 31.75$ and $16.25 > 16.15$, both chosen actions are strictly preferred, satisfying the Bellman optimality condition. (Note: $\rho \leq -3$ is necessary and sufficient for this policy to be optimal at $\gamma = 0.8$, and we chose -3.1 to ensure strict preference).

(d) Practical Meaning and the Case $\gamma = 1$

The discount factor γ represents the weight assigned to future rewards relative to immediate ones. It can denote time preference or, in equivalent models, the probability that the agent continues functioning for another step. It is an internal parameter rather than a physical transition probability of the environment.

At $\gamma = 1$, future rewards are completely undiscounted. Because this MDP is continuing and rewards accumulate indefinitely, the expected total rewards for all stationary deterministic policies diverge to infinity. Infinite values cannot provide a meaningful ranking, and the Bellman operator loses its contraction property. Therefore, $\gamma = 1$ requires alternative formulations, such as introducing a finite horizon, terminal states, or shifting to an average-reward objective.

Part 2: Optimal Policy ($\gamma = 0.8$)

Returning to the **original, unmodified MDP** with $\gamma = 0.8$, the updates are defined as:

$$Q^{(k+1)}(s, a) = R(s, a) + 0.8 \sum_{s'} P(s' | s, a) V^{(k)}(s'), \quad V^{(0)} = [0, 0].$$

(a) First Q-Value Update

Since all continuation values are initially zero, this yields the immediate rewards:

$$\boxed{Q^{(1)}(c, \text{eco}) = 7, \quad Q^{(1)}(c, \text{boost}) = 10, \quad Q^{(1)}(l, \text{eco}) = 0, \quad Q^{(1)}(l, \text{boost}) = 2.}$$

(b) Next Value Function

Taking the maximum over actions for each state gives:

$$\boxed{V^{(1)} = [\max(7, 10), \max(0, 2)] = [10, 2].}$$

(c) Value Iteration versus Policy Iteration

Value iteration (VI). Using $V^{(1)} = [10, 2]$, the next step is computed as:

$$\begin{aligned} Q^{(2)}(c, \text{eco}) &= 7 + 0.8(0.95 \cdot 10 + 0.05 \cdot 2) = 14.68, \\ Q^{(2)}(c, \text{boost}) &= 10 + 0.8(0.70 \cdot 10 + 0.30 \cdot 2) = 16.08, \\ Q^{(2)}(l, \text{eco}) &= 0 + 0.8(0.50 \cdot 10 + 0.50 \cdot 2) = 4.80, \\ Q^{(2)}(l, \text{boost}) &= 2 + 0.8(0.10 \cdot 10 + 0.90 \cdot 2) = 4.24. \end{aligned}$$

Taking the maximum over these actions results in:

$$V_{\text{VI}}^{(2)} = [16.08, 4.80].$$

Policy iteration (PI). The greedy policy derived from $V^{(1)}$ is $\pi^{(1)} = [\text{boost}, \text{eco}]$. Its exact evaluation requires solving the linear system for the infinite horizon:

$$\begin{aligned} V_c &= 10 + 0.8(0.7V_c + 0.3V_l), \\ V_l &= 0 + 0.8(0.5V_c + 0.5V_l). \end{aligned}$$

The second equation simplifies to $V_l = \frac{2}{3}V_c$. Substituting this into the first equation yields:

$$V_{\text{PI}}^{(2)} = V^{\pi^{(1)}} = \left[\frac{250}{7}, \frac{500}{21} \right] \approx [35.71429, 23.80952].$$

The resulting values differ significantly. Value iteration applies a single Bellman optimality backup per step, whereas exact policy evaluation calculates the total expected return of following the selected policy indefinitely.

(d) Optimal Policy

Evaluating the current policy based on its exact Q-values yields:

State	$Q(s, \text{eco})$	$Q(s, \text{boost})$
Charged	$737/21 \approx 35.09524$	$250/7 \approx 35.71429$
Low	$500/21 \approx 23.80952$	22

Since the current actions still possess the maximum values, the policy is greedy with respect to its own value function. Therefore, it is optimal:

$$\pi^*(c) = \text{boost}, \quad \pi^*(l) = \text{eco}, \quad V^* = \left[\frac{250}{7}, \frac{500}{21} \right].$$

Problem 2: Aster's Rescue Route—Comparing Active RL Methods

Assuming $\gamma = 0.8$. Note that the reward of 5 is received upon transitioning into T , and terminal states have a value of 0.

(a) Active ADP

Based on the observed transitions, action a has four occurrences, all leading to s_1 . Action b has five occurrences: three to T and two to s_1 . The maximum-likelihood estimates for the transition probabilities are:

Action	$\hat{P}(s_1 s_0, \cdot)$	$\hat{P}(T s_0, \cdot)$
a	1	0
b	$2/5$	$3/5$

Using the current estimate $U(s_1) = 4$, the model-based one-step lookahead values are:

$$\begin{aligned}\hat{Q}(s_0, a) &= 1(0 + 0.8 \cdot 4) = 3.2, \\ \hat{Q}(s_0, b) &= \frac{3}{5}(6 + 0.8 \cdot 0) + \frac{2}{5}(-2 + 0.8 \cdot 4) = 3.6 + 0.48 = 4.08.\end{aligned}$$

Since $4.08 > 3.2$, the greedy action is $\boxed{b}$. (Note: We use $U(s_1) = 4$ as instructed, without retroactively applying the newly observed reward of 5 at this stage).

(b) Monte Carlo Control

For the Monte Carlo update, the discounted return for the episode following (s_0, b) is:

$$G_0 = -2 + 0.8(5) = 2.$$

The sample-average estimate over all five complete returns is:

$$Q_{\text{MC}}(s_0, b) = \frac{6 + 6 + 6 + 2 + 2}{5} = \frac{22}{5} = 4.4.$$

(c) SARSA and Q-Learning

Both agents begin with $Q(s_0, b) = 2$ and $\alpha = 0.5$. Following a transition to s_1 with a reward of -2 , the behavior policy selects action d .

SARSA performs an on-policy update using the value of the selected action d :

$$\begin{aligned}Q_{\text{new}}(s_0, b) &= Q(s_0, b) + \alpha[r + \gamma Q(s_1, d) - Q(s_0, b)] \\ &= 2 + 0.5[-2 + 0.8(1) - 2] = \boxed{0.4}.\end{aligned}$$

Q-learning performs an off-policy update, using the maximum available action value at s_1 (which is $Q(s_1, c) = 4$), regardless of the behavior policy:

$$\begin{aligned}Q_{\text{new}}(s_0, b) &= Q(s_0, b) + \alpha[r + \gamma \max_{a'} Q(s_1, a') - Q(s_0, b)] \\ &= 2 + 0.5[-2 + 0.8(4) - 2] = \boxed{1.6}.\end{aligned}$$

(d) If the Behavior Policy Selects c Instead

If the behavior policy had selected c , SARSA would bootstrap from $Q(s_1, c) = 4$, yielding:

$$Q_{\text{new}}^{\text{SARSA}}(s_0, b) = 2 + 0.5[-2 + 0.8(4) - 2] = \boxed{1.6}.$$

The Q-learning update remains $\boxed{1.6}$ since the maximum Q-value at s_1 is unchanged. Because the selected action happens to be greedy, both updates coincide in this instance.

(e) Comparison of the Four Methods

The following table summarizes the operational differences among the four methods:

Method	What it learns	How experience improves the policy
Active ADP	A transition and reward model, computing values via planning.	Updates model probabilities, then applies expected Bellman backups to determine optimal actions.
Monte Carlo control	Action values derived directly from sampled complete returns.	Averages returns iteratively after episodes conclude, bypassing the need for a transition model.
SARSA	Action values specific to the current behavior policy.	Updates step-by-step using $r + \gamma Q(s', a')$ based on the actual next action, learning while executing behavior.
Q-learning	Optimal action values, independent of the behavior policy.	Updates step-by-step using $r + \gamma \max_{a'} Q(s', a')$, learning the optimal policy even during exploratory behavior.

(f) Why Exploration Is Necessary

Without exploration, an agent risks converging prematurely to suboptimal behavior. If early estimates inaccurately undervalue a rewarding action, a purely exploiting agent will never select it again to correct the estimate.

- **ADP** requires exploration to construct an accurate empirical model of transitions and rewards.
- **Monte Carlo** requires complete returns from diverse state-action pairs to construct accurate averages.
- **SARSA** and **Q-learning** require visitation of various states to accurately propagate Q-values backward. Even off-policy methods like Q-learning depend on the behavior policy to navigate sufficiently through the state space.

Employing strategies such as ϵ -greedy ensures the agent maintains a balance between refining known successful paths and discovering potentially better alternatives.

Problem 3 (Optional): Reinforcement Learning in the Real World

(a) Modeling Human Behavior and Decision Making

Reinforcement learning provides a robust computational framework for modeling human decision-making. We can map environmental contexts to states, choices to actions, and behavioral feedback to rewards. Decisions can then be modeled using prediction errors: when an outcome exceeds expectations, the internal value of that action increases, raising the probability of its future selection.

Importantly, human decision-making is not strictly model-free. Studies utilizing two-stage choice experiments have demonstrated that human behavior integrates both model-free cached values and model-based forward planning. Understanding this dual-process structure enables researchers to construct more accurate computational models of complex cognitive behaviors.

(b) Connection to Neuroscience

There is a well-documented connection between RL algorithms and neural mechanisms. Neurophysiological studies have demonstrated that the phasic responses of midbrain dopamine neurons closely parallel the Temporal Difference (TD) error:

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

Unexpected rewards trigger an increase in dopaminergic activity. Upon learning, this activity shifts from the reward to the predictive cue. Conversely, the omission of an expected reward results in a suppression of neural firing. Notably, reward and prediction error are distinct concepts: a highly expected reward generates little to no prediction error despite being inherently valuable.

This bidirectional relationship means that RL theory has aided the interpretation of neurobiological data, while neuroscientific evidence of interacting model-free and model-based systems inspires the development of hybrid AI architectures that combine learned values with explicit planning.

Generative AI Acknowledgement

Google Gemini and OpenAI ChatGPT were utilized as refinement aids during the preparation of this document. Their primary function was to provide high-level hints and suggestions, and ensure appropriate L^AT_EX formatting, enhancing the structural clarity and academic tone of the explanations. The authors comprehensively reviewed and manually verified all logical deductions, mathematical computations, and final solutions, assuming full responsibility for their correctness.